Thus, managing the hundreds of components and multiple licences and associated obligations presents challenges for mobile-application developers, handset manufacturers that use Android, and third-party companies that develop software components for device manufacturers.

Multi-source development and obligations in Android

The Android community includes Google, independent application developers, third-party companies that develop software for mobile and embedded devices, and manufacturers that adopt Android as the OS for a given mobile or embedded device. The multi-tiered ecosystem represents multi-source development at its best, yet it adds complexity to the Android platform.

Independent developers may contribute code under a variety of licences. Manufacturers may develop software IP to run on top of the Android OS, in addition to modifying and augmenting the Android code base to suit a particular hardware or software design. Commercial software development companies creating third-party applications may do all of the above, and add IP to Android components, use a variety of licences, and modify or augment the Android code base.

With this complexity comes the licence and IP management challenges. For instance: if the application software created/developed for the Android platform is not a final product, but is to be integrated as a component in a customer's final product (e.g., a mobile phone), then the company may be putting its customer's product at risk if the integration with the Android platform was not correctly managed.

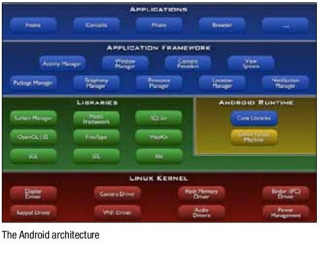
Handling the updates

Android code developed by Google includes changes outside the Linux kernel, which created an initial fork. From 2009's V 1.5 'Cupcake' release to today's V 2.2 'Froyo' release, bug fixes, enhancements and patches have been added to the main project. Development occurs at such a rapid pace that there is an acknowledged large backlog of patches from Android that are headed back upstream to the Linux kernel—which keeps the Linux kernel and Android's fork out of sync.

"Therefore, the Android OEMs and device builders must continuously update their local copies with the latest ongoing developments, to stay in position to release their products immediately after Android releases. These changes need to be reviewed and tested for compatibility—and assessed for compliance. With substantial development going on in parallel, it is imperative to have a strategy to manage the complexity, and to identify and approve the changes being incorporated in products," advises Sharma.

The obligations

The Android project contains over 19 different licence types in over 185 different projects (or components). Assuming each licence is broken down into its respective obligations,



The Android architecture

and those obligations are assigned to each component usage, over 1,700 obligations come into play. Of these, over 1,000 are legal-type obligations, while around 700 are developer-type obligations. Fortunately, most companies don't need to confirm compliance to all 1,700 obligations. Legal obligations typically involve accepting a disclaimer of warranty, limiting liability, protection of trademarks, or other items that generally do not add work for the developer.

"However, even the most permissive licence typically has an obligation of acknowledgement, and other obligations (marking modification, redistribution requirements, documentation requirements, etc) that can add work to the software developer's backlog of tasks. And, developer-level obligations often need to be managed, not at the component level, but at the file level. In a highly dynamic environment like Android development, where files are frequently updated from the Web repositories, keeping track of all these obligations can be daunting," explains Sharma.

Using Black Duck's Android Fast Start makes managing all these complexities related to source-code reviews a straightforward process, he affirms.

The software allows your software-development activity to benefit from open source, and manage the complexity of Android, while allowing legal, IT, security and export personnel to access timely and relevant information to effectively manage business risks. The tool can also address a broad set of management, compliance, and security problems that surface when open source components are used on a significant scale in software development. So, if you are in software development, the best route to write safe code and minimise the complexity that may arise from the use of open source would be to look for a tool like Black Duck. END

By: Vandana Sharma

The author is an assistant editor at the EFY Group. Apart from writing on subjects like entrepreneurship and IT for businesses, she loves to read on a range of topics, which include religion and spirituality. She enjoys 'meaningful' fiction, too.